

Documentación del Proyecto Videoconsola LPC2106

Desarrollo Práctico de Sistemas Electrónicos

Alonso Sandoval Martínez
Luis F. Sánchez Turrión
Mario Medrano Paredes

21 de mayo de 2025

Resumen

Este trabajo presenta el diseño, implementación y verificación de una videoconsola retro portátil basada en el microcontrolador LPC2106, desarrollada en el marco de la asignatura Desarrollo Práctico de Sistemas Electrónicos. La consola integra un módulo TFT con un panel táctil resistivo, dos crucetas para los controles, un amplificador estéreo con dos altavoces, un motor vibrador para realimentación háptica, conector USB y alimentación mediante batería de polímero de litio con circuito de carga integrado. En la vertiente hardware se llevó a cabo la captura esquemática en Proteus, el diseño y fabricación de la PCB en dos mitades, la soldadura manual de componentes y el montaje de la carcasa impresa en 3D con resina. En paralelo, se desarrolló el firmware en C para controlar GPIO, ADC, drivers de pantalla, audio y vibración, así como la adaptación del videojuego clásico Pacman al proyecto. Tras la simulación, depuración y caracterización eléctrica y térmica, la consola demostró un funcionamiento estable y autónomo, validando así la metodología de diseño global desde la especificación hasta la puesta en marcha.

Palabras clave

Videoconsola portátil, LPC2106, diseño de circuitos electrónicos, impresión 3D, firmware en C

Índice general

1. Introducción 3

2. Planificación 4

2.1. Diagrama de Gantt 4

2.2. Control de cambios 6

3. Hardware 8

3.1. Diseño electrónico y captura esquemática 8

3.1.1. Importación y creación de componentes 8

3.1.2. Posicionado de componentes. Esquemas electrónicos 8

3.1.3. Bill Of Materials (BOM) 13

3.1.4. Análisis eléctrico y análisis térmico 16

3.2. Diseño de la placa de circuito impreso 17

3.2.1. Importación y creación de huellas 17

3.2.2. Posicionado de huellas. Layout PCB 18

3.2.3. Generación de ficheros gerber 21

3.3. Fabricación y montaje 21

3.3.1. Recepción y revisión 21

3.3.2. Montaje inicial. Soldado de componentes 21

3.3.3. Verificación y caracterización del hardware 22

3.3.4. Montaje final 23

4. Software 24

4.1. Desarrollo del firmware. MCU LPC2106 y simulador 24

4.2. Controladores 25

4.2.1. Motor vibrador 25

4.2.2. Audio 26

4.3. Videojuego: Pacman clásico 29

5. Conclusiones y líneas futuras 33

Referencias 34

1. Introducción

En el contexto de la asignatura Desarrollo Práctico de Sistemas Electrónicos [1], el presente proyecto aborda el diseño, la implementación y la verificación de una videoconsola portátil desde sus fases iniciales hasta su montaje final y puesta en funcionamiento [2]. La creciente demanda de dispositivos electrónicos integrados, junto con la evolución de microcontroladores de bajo coste y pantallas de alta resolución, motiva la creación de sistemas compactos capaces de ofrecer experiencias multimedia completas. En este caso, la elección del microcontrolador ARM7 LPC2106 [3] permite explorar tanto la parte hardware con Proteus [4] (captura esquemática, diseño de PCB, manufactura y ensamblaje) como la vertiente software (desarrollo de drivers y del videojuego clásico Pacman adaptado al dispositivo).

El principal objetivo de este trabajo consiste pro tanto en entregar una plataforma funcional y autónoma de juego, que incorpore un módulo TFT de 800×480 píxeles con panel táctil resistivo, un amplificador estéreo con dos altavoces, un sistema de realimentación háptica mediante motor vibrador, y autonomía eléctrica a través de batería de polímero de litio con circuito de carga integrado. Para ello, se han definido metas específicas: especificar los requerimientos, capturar y validar el esquema electrónico, diseñar y fabricar la placa de circuito impreso, desarrollar el firmware que controle cada subsistema (GPIO, ADC, audio, pantalla y vibración), implementar la lógica de juego y, finalmente, verificar el correcto funcionamiento eléctrico, térmico y funcional de la consola.

El presente informe se estructura en tres secciones principales. En la primera se presenta la planificación del proyecto, incluyendo el diagrama de Gantt, y los ficheros para el control de cambios o de versiones. La segunda profundiza en el diseño electrónico y la captura esquemática. A continuación, se describe el diseño de la placa de circuito impreso y la generación de los ficheros Gerber, así como el proceso de fabricación y montaje de componentes. La tercera sección aborda la realización del firmware, la simulación y la depuración. Finalmente, se exponen las conclusiones, la consecución de los objetivos presentados, y las posibles líneas de mejora y ampliaciones futuras del proyecto.

2. Planificación

La sección planificación involucra aquellas partes relacionadas con la gestión y el seguimiento del proyecto, englobando no solo la elaboración y mantenimiento del diagrama de Gantt, sino también el control de cambios o de versiones. De este modo, se garantiza una visión global y actualizada de los avances realizados. El seguimiento riguroso de las tareas y la documentación asociada facilitan asimismo la transparencia en la evolución del proyecto y en la toma de decisiones.

2.1. Diagrama de Gantt

El diagrama de Gantt (ver Figura 2.1) se elaboró con *GanttProject* [5], una herramienta de planificación de proyectos de código abierto desarrollada en Java que permite definir tareas, establecer dependencias, asignar recursos y actualizar el grado de avance de forma interactiva. Para la generación inicial del diagrama, se empleó un calendario proporcionado por el profesor, que incluía todos los festivos del año 2025 en la región de Castilla y León y que fue creado con el soporte del modelo de inteligencia artificial Qwen de Alibaba.

La primera versión del diagrama se construyó tomando como base la estructura y las fechas de los apuntes de la asignatura, si bien fue necesario reajustarlas para adecuarlas al curso académico 2024-2025 y garantizar la coherencia con las sesiones lectivas y los plazos de entrega.

A lo largo de las dieciocho semanas que abarcan desde el 10 de febrero de 2025 hasta el 27 de mayo de 2025, se definieron más de treinta actividades, desde la especificación inicial hasta la realización del informe final, cubriendo diseño electrónico, fabricación de la PCB, desarrollo de software y verificación de hardware. Cada tarea dispone de fecha de inicio y fin, y semanalmente se actualizó su porcentaje de cumplimiento para reflejar tanto hitos parciales como tareas finalizadas.

Finalmente, se exportó el diagrama a formato PDF, lo que facilitó su impresión y análisis durante las reuniones de seguimiento semanales y las entregas.

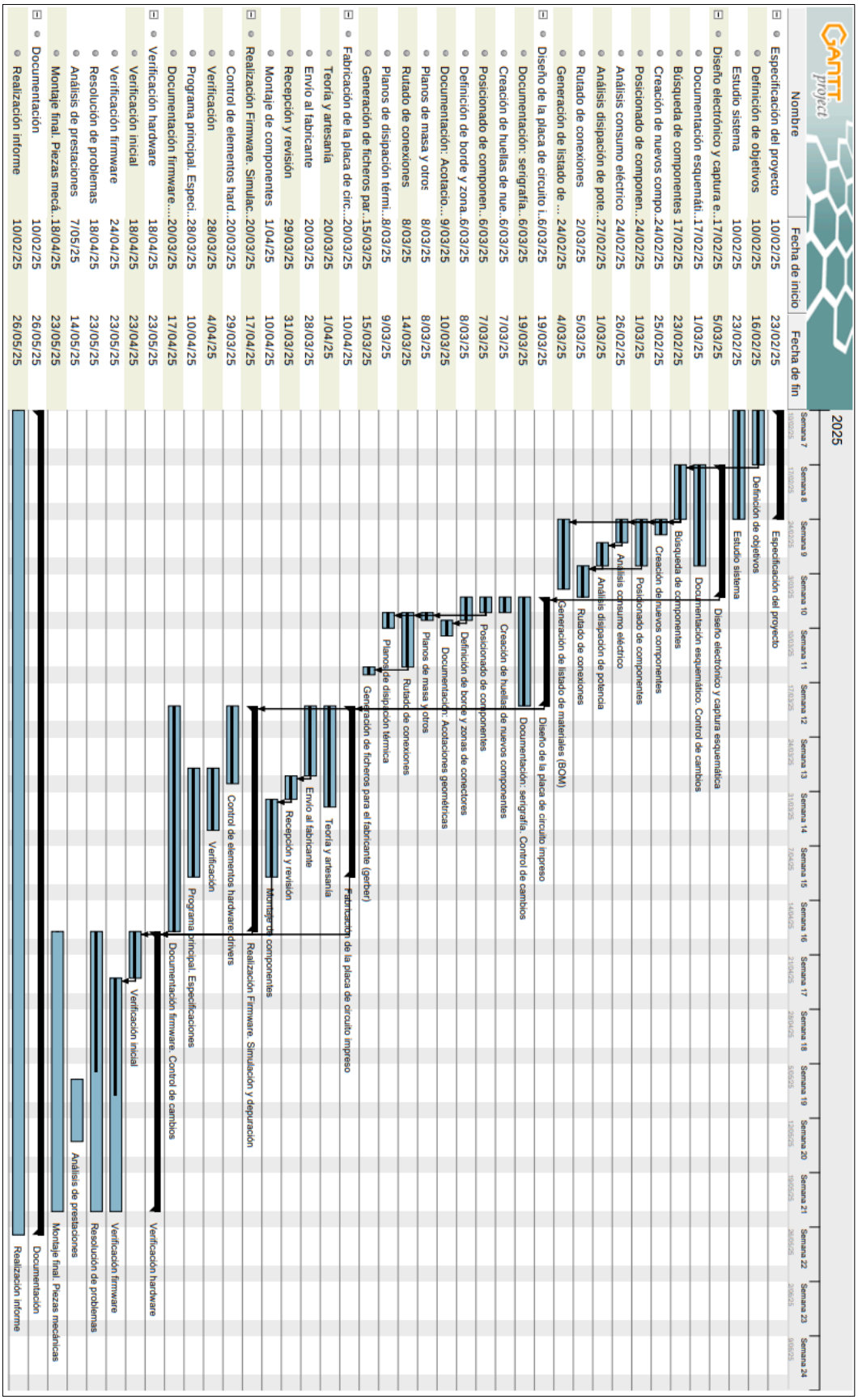


Figura 2.1. Diagrama de Gantt del proyecto

2.2. Control de cambios

El seguimiento de las versiones y modificaciones del proyecto se gestionó mediante ficheros de texto plano (.txt). Cada semana, en las sesiones programadas los lunes y los miércoles, se registró en cada uno de los cuatro ficheros correspondientes (planificación, hardware, software y documentación) las incidencias, avances y correcciones realizadas, aprovechando para ello los mensajes descriptivos incluidos en los *commits* del repositorio GitHub correspondiente a nuestro proyecto para garantizar la trazabilidad de cada contribución hecha por cada uno de los miembros. Este mecanismo permitió centralizar en un único documento el histórico de cambios, facilitando tanto la revisión de decisiones pasadas como la identificación de posibles desviaciones respecto al plan original.

Los ficheros de control de cambios (ver Figuras a continuación) siguieron un formato de entradas fechadas con día, mes y año, seguidas de una etiqueta con el tipo de cambio y una breve descripción de la labor realizada. Este enfoque semántico y cronológico facilitó la generación de informes posteriormente.

```
10/02/2025
- (Añadido): Grupos de trabajo creados.
- (Añadido): Diagrama de Gantt y especificación del proyecto.

12/02/2025
- (Añadido): Creación del proyecto en Proteus.
- (Modificado): Calendario del diagrama de Gantt con los festivos del curso 2024-2025.

17/02/2025
- (Añadido): Control de versiones creado.
```

Figura 2.2. Control de cambios para la planificación.

```
09/04/2025
- (Añadido): Análisis del software y las especificaciones requeridas.

28/04/2025
- (Añadido): Inicio del desarrollo del videojuego. Ideas iniciales.

30/04/2025
- (Modificado): Elección de desarrollar Pacman.
- (Añadido): Planteamiento de la lógica del videojuego.

05/05/2025
- (Añadido): Diseño de gráficos y ficheros .xpm.

07/05/2025
- (Corregido): Gráficos arreglados.
- (Añadido): Implementación de la lógica del videojuego.

12/05/2025:
- (Corregido): Sistema de puntuación y movimiento arreglado.
- (Añadido): Drivers para la reproducción de audio.

14/05/2025:
- (Añadido): Drivers para el funcionamiento del motor vibrador.

19/05/2025:
- (Corregido): Bugs arreglados.

21/05/2025:
- (Añadido): Videojuego finalizado. Firmware final cargado en la consola.
```

Figura 2.3. Control de cambios para el software.

```
17/02/2025
- (Añadido): Esquemático de la hoja 1 completado.

19/02/2025
- (Añadido): Esquemáticos de las hojas 2 y 3 completados.

24/02/2025
- (Añadido): Esquemático de la hoja 4 completado.
- (Modificado): Nuevos textos y logo en la caja informativa del esquemático.
- (Corregido): Referencias y paquetes corregidos en el esquemático.
- (Añadido): Diseño de la PCB iniciado.

26/02/2025
- (Añadido): Primer borrador del BOM.
- (Añadido): Búsqueda de nuevas huellas de componentes.
- (Corregido): Huellas de la PCB modificadas.

03/03/2025
- (Añadido): Nuevas huellas de componentes creadas.

05/03/2025
- (Añadido): Posicionamiento inicial de componentes en el PCB.

10/03/2025
- (Corregido): Ajustes en el esquemático.
- (Corregido): Corrección de huellas de componentes.
- (Modificado): Desplazamiento de componentes en la PCB.

12/03/2025
- (Añadido): Rutado de conexiones.

17/03/2025
- (Corregido): Rutado mejorado.
- (Añadido): Planos de masa creados.
- (Añadido): Generación de ficheros Gerber.

19/03/2025
- (Añadido): Corte de la PCB.
- (Añadido): Soldadura de circuitos integrados.

24/03/2025
- (Añadido): Soldadura de resistencias y condensadores.

26/03/2025
- (Añadido): Soldadura del resto de componentes.

31/03/2025
- (Añadido): Soldadura finalizada.
- (Añadido): Revisión de la carcasa impresa en 3D.

02/04/2025
- (Añadido): Verificación de componentes. Batería, pantalla y comunicación con el PC
comprobadas.

07/04/2025:
- (Corregido): Resoldar de algunos integrados.
```

Figura 2.4. Control de cambios para el hardware.

```
07/05/2025
- (Añadido): Inicio de la documentación del proyecto.

12/05/2025:
- (Añadido): Introducción y resumen del proyecto.

14/05/2025:
- (Añadido): Sección sobre el diagrama de Gantt y el control de versiones.

19/05/2025:
- (Añadido): Sección sobre el hardware y el montaje.
- (Añadido): Capturas de los esquemáticos, el BOM, la PCB y el montaje.

21/05/2025:
- (Añadido): Sección sobre el software y el desarrollo del videojuego.

26/05/2025:
- (Corregido): Imágenes actualizadas.
- (Añadido): Explicación sobre los drivers de audio y del motor de vibración.

28/05/2025:
- (Añadido): Fotografías del montaje final.
- (Corregido): Revisión de la documentación y entrega.
```

Figura 2.5. Control de cambios para la documentación

3. Hardware

Esta sección aborda el diseño e implementación de la parte física de la consola portátil, desde la concepción del esquema electrónico hasta el montaje y verificación final de la placa. En primer lugar, se detallará el proceso de captura esquemática, incluyendo la importación y creación de componentes, el posicionamiento en el diagrama y la generación del listado de materiales junto con los análisis eléctrico y térmico correspondientes. A continuación, se describirá el diseño de la placa de circuito impreso, cubriendo la creación de huellas, el *layout*, y la exportación de ficheros Gerber para fabricación. Por último, se explicarán las fases de montaje de componentes y las pruebas de funcionamiento, así como la resolución de incidencias y el ensamblaje final.

3.1. Diseño electrónico y captura esquemática

3.1.1. Importación y creación de componentes

La captura de los símbolos esquemáticos se llevó a cabo principalmente a través de la integración de Proteus [4] con los repositorios en línea SnapEDA [6] y SamacSys [7], así como mediante el servicio ECAD gratuito de Mouser. Gracias al contrato de actualización de Proteus, se importaron directamente desde dichos repositorios los modelos 2D y 3D de componentes estándar, lo que agilizó notablemente el proceso de diseño al disponer de bibliotecas actualizadas y validadas por los proveedores. De manera complementaria, se utilizó el *Library Loader* de Mouser para obtener símbolos y huellas adicionales, arrastrándolos directamente sobre el entorno de Proteus tras su descarga desde la plataforma de *Mouser Free ECAD Service*.

En su mayoría, los componentes seleccionados fueron de tecnología SMD, usándose puntualmente algunos de inserción (por ejemplo conectores o pines). En algunos de los componentes, como por ejemplo el *slot* para la tarjeta microSD, no existían las huellas con la misma silueta exacta que la presentada en los ejemplos, de modo que se seleccionaron símbolos genéricos cuyo patillaje coincidía con el componente real, lo que resultó suficiente para realizar los esquemáticos y el rutado.

Por otro lado, aquellos elementos no disponibles en ninguna biblioteca, como por ejemplo el potenciómetro para regular el volumen, se modelaron desde cero en Proteus, definiendo manualmente los pines, el encapsulado y los parámetros eléctricos necesarios para garantizar una representación fiel y un correcto análisis posterior.

3.1.2. Posicionado de componentes. Esquemas electrónicos

El posicionamiento de componentes y el desarrollo de los esquemáticos se realizó distribuyendo las cuatro hojas de proyecto por módulos funcionales para maximizar la claridad y facilitar el seguimiento de señales críticas:

1) Microcontrolador, pantalla táctil TFT y ranura para la tarjeta microSD

2) USB, alimentación y batería
3) Botones y motor vibrador
4) Sistema de audio (altavoces, amplificador, etc.)

En cada hoja se añadieron puntos de prueba (*test points*) en todas las líneas de alimentación (3V3, 1V8, 3V7, GND), datos y control, lo que permite llevar a cabo comprobaciones eléctricas o de señal sin necesidad de acceso directo a los pines del encapsulado. De igual modo, en todos los símbolos se hizo visible la referencia (*reference*), el valor (*value*) y el nombre de huella (*PCB Package*) para obtener toda la información de los componentes rápidamente. Además, las etiquetas de red (net-labels) identifican de forma inequívoca cada conexión entre hojas, evitando errores de cableado durante el posterior rutado.

La primera hoja, denominada *Microcontroller & TFT/TP/BL*, integra el LPC2106 (U1) con sus condensadores de desacoplo de 100 nF (C3–C5) entre V3_3 V, V1_8 V y tierra. El cristal de 14.7456 MHz (X1) y sus capacitores de carga (C1–C2) se colocaron adyacentes al microcontrolador para minimizar trazas y ruido. El bus de datos de 16 bits DB0–DB15 queda posicionado junto a la pantalla TFT (M4), mientras que el panel táctil SPI (TP_CS, MISO, MOSI, SCK) se sitúa en paralelo pero claramente diferenciado. Para multiplexar DB0–DB7 entre el TFT y los ocho pulsadores de las crucetas, se incluyeron resistencias de 10 kΩ (R20–R27), de modo que al activar un botón no se produzcan cortocircuitos durante la comunicación con la pantalla. El conector hembra J12, idéntico al módulo TFT/TP, facilita el desmontaje y reutilización del display.

La segunda hoja, *USB/Power/Battery*, agrupa el conector Mini-USB B (J2), el filtro LC de entrada (L1–C6) y el cargador de batería MCP73831 (U3) con sus condensadores de entrada y salida (C10–C11) y la resistencia de programación R8. Junto a ellos, el interruptor de selección de fuente SW1 permite alternar entre alimentación por USB (5 V) o por batería (pin BAT1). A continuación se posicionaron los reguladores MCP1725-ADJ en cascada (U4 para 3.3 V y U2 para 1.8 V), con sus divisores de tensión (R3/R4 y R6/R7) y condensadores de desacoplo (C7–C9). Las etiquetas 5V, 3V3M y 1V8M enlazan esta hoja con el resto de los módulos.

En la tercera hoja, *Buttons/Vibrator*, se agrupan los ocho interruptores de las crucetas con sus resistencias de pull-up (R31–R38) y los conectores de inter-placa J6/J7, que permiten segmentar el diseño en dos PCBs. El circuito de vibración incluye un MOSFET de P-canal (Q1) con su pull-up R13 y diodo de protección D2, así como el punto de test VIBRADOR, ubicado estratégicamente para facilitar la verificación de la señal de activación.

La última hoja, *Audio & USB-UART,* dispone el puente USB-UART CY7C65213 (U8) con su condensador de desacoplo C20 y las resistencias R14–R17 para RESET y BOOT, necesarias para gestionar el modo normal o *bootloader* del microcontrolador. A su lado, el amplificador de audio clase D PAM8403 (U7) se rodea de condensadores de desacoplo (C16–C21), filtros pasa-bajos y condensadores de acoplo (C22–C27) antes de los altavoces LS1 y LS2. Cada señal de control y audio (AUDIO_L, AUDIO_R, SPKL+,

SPKR+) está etiquetada, lo que garantiza la trazabilidad desde el esquema hasta el *layout* de PCB. Este planteamiento modular y documentado agiliza el rutado, minimiza interferencias y facilita la fase de pruebas en laboratorio.

A continuación se presenta el resultado final del esquemático para todas las partes del circuito electrónico de nuestro proyecto. La parte inferior derecha de cada hoja contiene un cuadro de texto con la información general sobre el proyecto, los autores, la fecha y el número de revisión, así como el título de cada página del diseño.

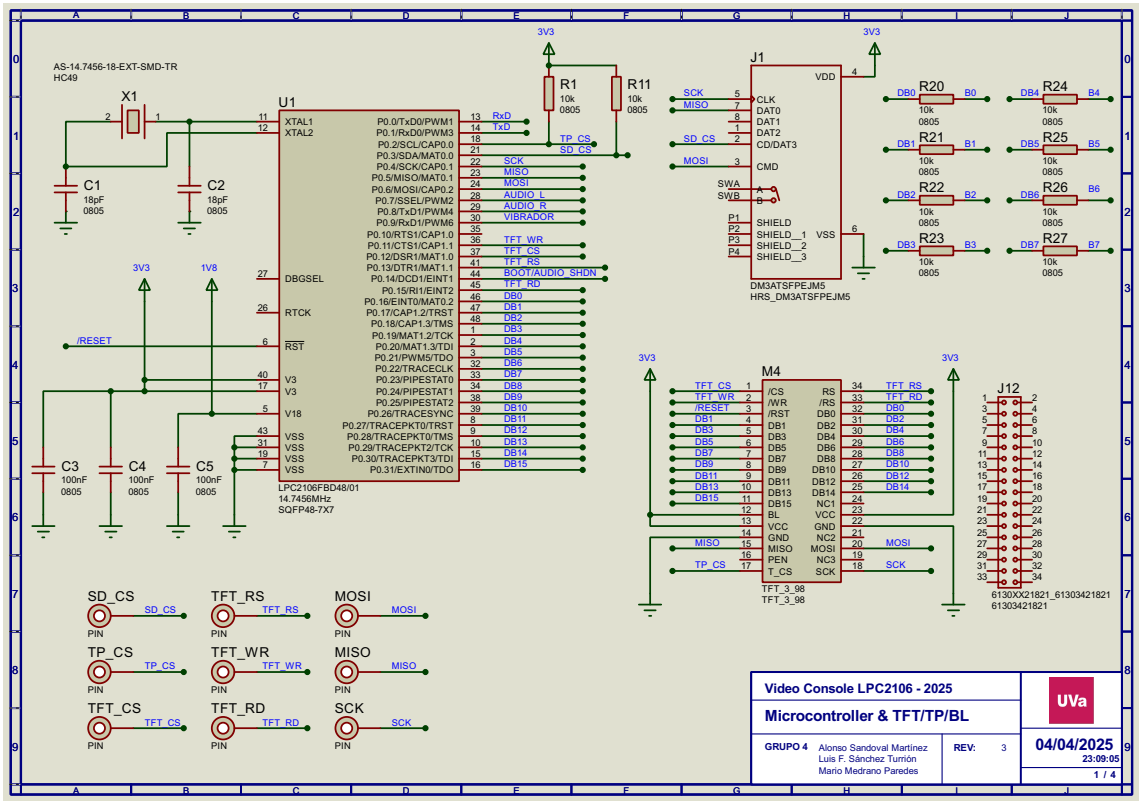


Figura 3.1. Hoja "Microcontroller & TFT/TP/BL" del esquemático

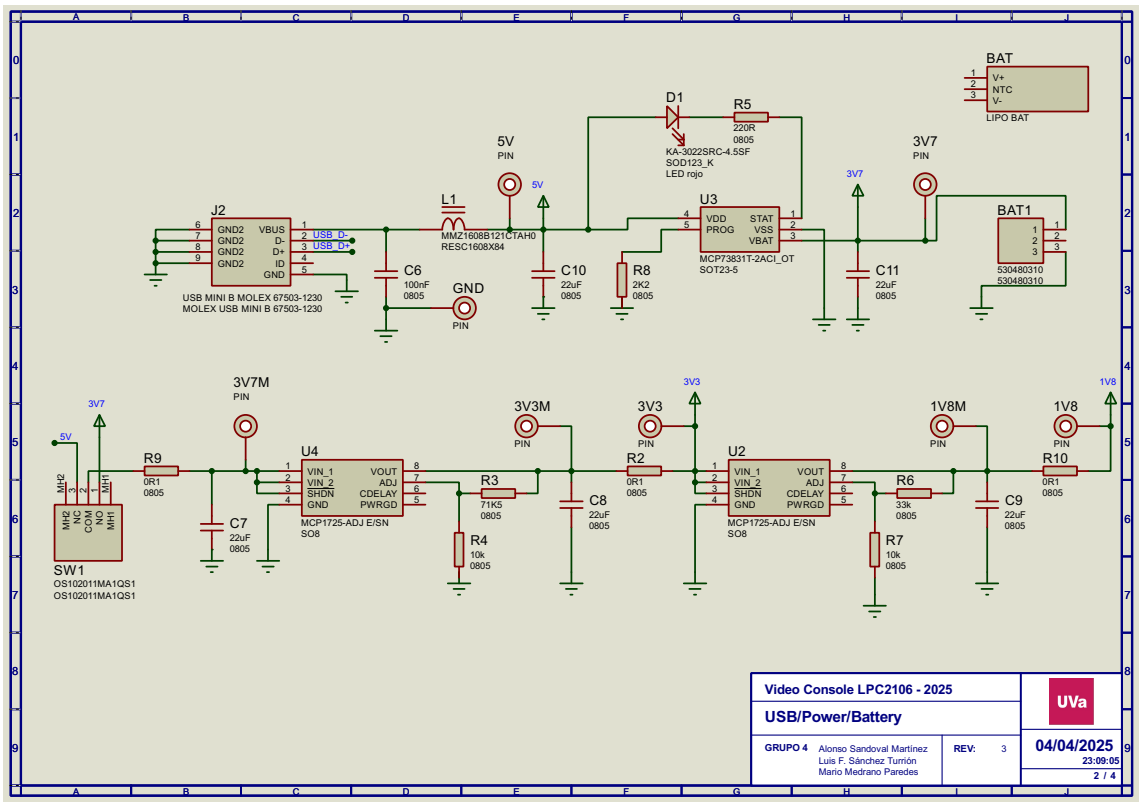


Figura 3.2. Hoja "USB/Power/Battery" del esquemático

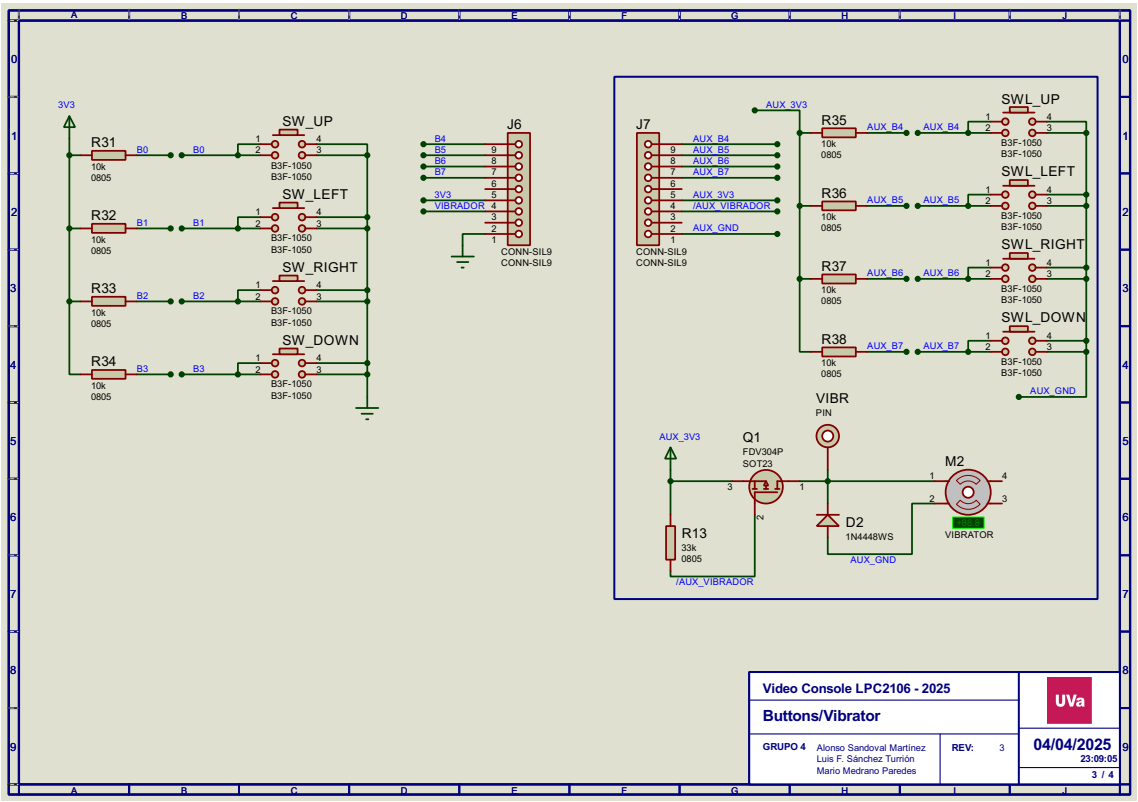


Figura 3.3. Hoja "Buttons/Vibrator" del esquemático

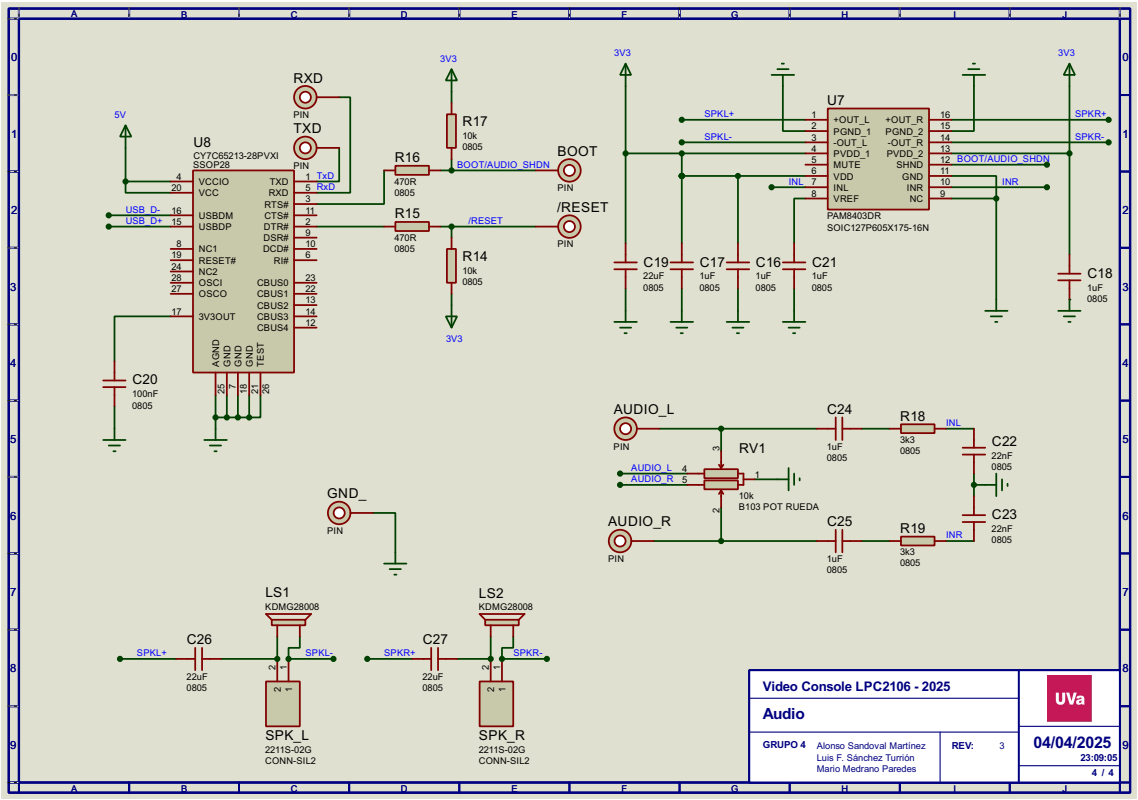


Figura 3.4. Hoja. "Audio" del esquemático

3.1.3. Bill Of Materials (BOM)

El *Bill of Materials* (BOM) o listado de materiales es un documento estructurado que recopila todas las piezas, componentes y suministros necesarios para la fabricación del proyecto, indicando para cada uno su cantidad, referencia de proveedor, encapsulado y coste unitario. Su elaboración nos facilitó la planificación de compras, el control de inventario y la estimación precisa del coste total. Además, al disponer de un BOM completo y actualizado, se agiliza la gestión de proveedores, se reducen problemas de desabastecimiento y se asegura la trazabilidad de cada elemento desde la fase de diseño hasta el montaje final.

En nuestro proyecto, el listado de materiales se generó a partir de la plantilla por defecto de Proteus, adaptada para incluir los campos esenciales: Quantity, References, Value, PCB Package, Stock code (Farnell code) y Cost (€/unidad). La plantilla resultante se guardó en un fichero con extensión .bomt. Para cada componente se asoció la referencia de Farnell, consultando siempre en la página de dicho proveedor [8] el precio para pedidos de diez o más unidades, ya que se realizó un pedido conjunto con los componentes para todos los grupos de alumnos de la asignatura. En los casos en que no existía código Farnell se recurrió al proveedor AliExpress para estimar el coste, ya que es una de las plataformas más usadas para este propósito. Se empleó la coma como separador decimal para ajustarse a la normativa española y se excluyeron del BOM aquellos elementos sin precio o irrelevantes (por ejemplo, los pines de test).

El *Bill Of Materials* se organizó por familias de componentes: módulos, condensadores, resistencias, circuitos integrados, transistores, diodos y miscelánea. A modo de ejemplo, en la sección de circuitos integrados aparece el microcontrolador U1 (LPC2106FBD48/01, encapsulado SQFP48-7x7) con código Farnell 1832283 y coste unitario de 15,04 €, mientras que por ejemplo los condensadores de desacoplo 100 nF (C3–C6, C20) en SMD 0805 no llevan código al ser genéricos tienen y un precio de 0,0182 € por unidad. En la sección de miscelánea destacan el conector USB Mini-B Molex 67503-1230 (Farnell 2135162) a 0,913 € y los conectores de inter-placa CONN-SIL9 (códigos Farnell 2847166 y 2847198) a 0,056 € y 0,0427 € respectivamente.

El coste total de todos los componentes ascendió a 67,4838 €, distribuido entre las distintas categorías según las cantidades requeridas para la fabricación de la consola portátil. El desglose completo puede consultarse en la Figura 3.5 y la Figura 3.6.

Bill Of Materials for DPSE_grupo4

Design Title	DPSE_grupo4
Author	Grupo 4
Document Number	
Revision	
Design Created	lunes, 17 de febrero de 2025
Design Last Modified	lunes, 10 de marzo de 2025
Total Parts In Design	116

2 Modules

Quantity	References	Value	PCB Package	Farnell code	Price per unit
1	M2	VIBRATOR	VIBRATOR	Aliexpress	1,089€
1	M4	TFT_3_98	TFT_3.98	Aliexpress	15,17€
Sub-totals:					16,2590€

23 Capacitors

Quantity	References	Value	PCB Package	Farnell code	Price per unit
2	C1-C2	18pF	0805		0,0182€
5	C3-C6,C20	100nF	0805		0,0182€
8	C7-C11,C19,C26-C27	22uF	0805		0,0182€
6	C16-C18,C21,C24-C25	1uF	0805		0,0182€
2	C22-C23	22nF	0805		0,0182€
Sub-totals:					0,4186€

34 Resistors

Quantity	References	Value	PCB Package	Farnell code	Price per unit
22	R1,R4,R7,R11,R14,R17,R20-R27,R31-R38	10k	0805		0,0125€
3	R2,R9-R10	0R1	0805		0,0125€
1	R3	71K5	0805		0,0125€
1	R5	220R	0805		0,0125€
2	R6,R13	33k	0805		0,0125€
1	R8	2K2	0805		0,0125€
2	R15-R16	470R	0805		0,0125€
2	R18-R19	3k3	0805		0,0125€
Sub-totals:					0,4250€

6 Integrated Circuits

Quantity	References	Value	PCB Package	Farnell code	Price per unit
1	U1	LPC2106FBD48/01	SQFP48-7X7	1832283	15,04€
2	U2,U4	MCP1725-ADJ E/SN	SO8	1834878	0,658€
1	U3	MCP73831T-2ACI_OT	SOT23-5	1332158	0,754€
1	U8	CY7C65213-28PVXI	SSOP28	2767866	1,84€
1	U15	PAM8403DR	SOIC127P600X126-16N	3373823	0,672€
Sub-totals:					19,6220€

1 Transistors

Quantity	References	Value	PCB Package	Farnell code	Price per unit
1	Q1	FDV304P	SOT23	9846123	0,387€
Sub-totals:					0,3870€

2 Diodes

Figura 3.5. Bill Of Materials, página 1

Quantity	References	Value	PCB Package	Farnell code	Price per unit
1	D1	KA-3022SRC-4.5SF	SOD123_K	1142617	0,4€
1	D2	1N4448WS	SOD-323	1843752	0,0883€
Sub-totals:					0,4883€

48 Miscellaneous

Quantity	References	Value	PCB Package	Farnell code	Price per unit
25	1V8,1V8M,3V3,3V7, 3V3M,3V7M,5V, AUDIO_L,AUDIO_R, GND,GND_,J10-J11, MISO,MOSI,RXD, SCK,SD_CS, TFT_CS,TFT_RD, TFT_RS,TFT_WR, TP_CS,TXD,VIBR	PIN	PIN		
1	BAT	LIPO BAT	BAT	RS: 144-9405	14,22€
1	BAT1	530480310	530480310	9732918	0,186€
1	J1	DM3ATSFPEJM5	HRS_DM3ATSFPEJM5	2427719	1,84€
1	J2	USB Mini B Molex 67503-1230	MOLEX USB MINI B 67503-1230	2113554	0,913€
1	J6	CONN-SIL9	CONN-SIL9-MALE-90	2847166	0,056€
1	J7	CONN-SIL9	CONN-SIL9-FEMALE-90	2847198	0,0427€
1	J12	6130XX21821_61303421821	61303421821	3583802	1,15€
1	L1	MMZ1608B121CTAH0	RESC1608X84	2800823	0,046€
2	LS1-LS2	SPEAKER	CONN-SIL2	1502732	1,43€
1	RV1	10k	B103 POT RUEDA	Aliexpress	3,99€
2	SPK_L,SPK_R	2211S-02G	HDRV2W64P0X254_1X2_508X254X874P	1593411	0,0131€
1	SW1	OS102011MA1QS1	OS102011MA1QS1	2435103	0,401€
8	SWL_DOWN, SWL_LEFT, SWL_RIGHT, SWL_UP, SW_DOWN, SW_LEFT, SW_RIGTH,SW_UP	B3F-1050	B3F-1050	176433	0,498€
1	X1	AS-14.7456-18-EXT-SMD-TR	HC49	4165867	0,169€
Sub-totals:					29,8839€

Totals: 67,4838€

lunes, 10 de marzo de 2025 19:30:17

Figura 3.6. Bill Of Materials, página 2

3.1.4. Análisis eléctrico y análisis térmico

Análisis eléctrico

El análisis eléctrico tiene como objetivo cuantificar el consumo de corriente de los distintos subsistemas de la consola y dimensionar adecuadamente las fuentes de alimentación, las pistas de la PCB y los reguladores empleados para asegurar la fiabilidad global del sistema. Para ello, se han seleccionado únicamente los componentes cuyo consumo resulta significativo en comparación con el conjunto (microcontrolador, pantalla TFT táctil, motores, amplificador de audio, reguladores de tensión, etc.) y se ha estimado el tiempo de activación de cada uno durante el uso típico (ejecución del firmware, refresco de pantalla, lecturas de panel táctil, reproducción de sonido y vibración). Esto se ha hecho ya que el aporte de otros elementos (p.ej. resistencias de pull-up o pines de test) es despreciable en el cómputo total. Los valores de corriente se han extraído de las hojas de datos (*datasheets*) de cada dispositivo, considerando tanto las corrientes de reposo (*quiescent current*) como los picos de consumo en condiciones de máxima actividad (por ejemplo, refresco de la pantalla a todo color o activación simultánea del motor y el audio).

En la Figura 3.7 se agrupan estas corrientes por rail de tensión (5 V, 3.7 V, 3.3 V, 1.8 V) y se calcula el consumo total en condiciones de "peor caso" para garantizar que los reguladores lineales y el filtro de entrada soporten sin problemas la demanda máxima de corriente. A partir de estos datos, se dimensionan los anchos de pista de la PCB según las recomendaciones de corriente-anchura de cobre y se comprueban las caídas de tensión admisibles en cada trazado.

Power Calculations					
Component	**Conditions**	**Voltage(Volt)**	**Current(A)**	**Power(W)**	
LPC2106	(@60MHz)	1.8	0.04	0.072	
	Idle 7mA	1.8	0.007	0.0126	
	Power-down 10uA	1.8	0.01	0.018	
			TOTAL: 57mA	TOTAL= 0,1026W	
MCP1725-ADJ	3V3-> 1V8	3.3	0.057	0.1881	
	Quiescent current 220uA	3.3	0.00022	0.000726	
			TOTAL=57.22mA	TOTAL=0.1888W	
TFT 3.98		3.3	0.02	0.066	
Vibrator DC Motor		3.3	0.07	0.231	Uso 50% del tiempo--> Power= 0.1155
Auido Amplifier		3.3	0.185	0.6105	
			TOTAL=275mA	TOTAL=0,9075W	
MCP1725-ADJ	3V7-> 3V3	3.7	0.275	1.0175	
	Quiescent current 220uA	3.7	0.22	0.814	
			TOTAL=297mA	TOTAL=1.8315W	
Red LED		5	0.0065	0.0325	
USB-serial		5	0.015	0.075	
MPC738321		5	0.0015	0.0075	
			TOTAL=23mA	TOTAL=0,115W	

Figura 3.7. Análisis eléctrico

Análisis térmico

El análisis térmico persigue evaluar la disipación de potencia en los componentes activos y estimar su elevación de temperatura para asegurar que la temperatura de unión

(Tj) no supere los límites máximos definidos en sus hojas de datos. Para ello se calcula la potencia disipada en cada elemento crítico (por ejemplo, P = (VIN–VOUT) * I en los reguladores lineales o P = I² * R en MOSFETs y resistencias de paso) y se aplica la resistencia térmica junction-to-ambient (ΘJA) correspondiente. Mediante la expresión ΔT = P·ΘJA se obtiene la subida de temperatura sobre la temperatura ambiente, permitiendo verificar que Tj permanece en zona segura bajo la carga máxima estimada. En los casos en que el resultado se acerca al límite, se contemplaría la ampliación de planos de masa o la inclusión de áreas de cobre adicionales como disipadores pasivos. Al igual que en el caso anterior, se contempla el "peor caso" para las condiciones de cada uno de los componentes.

Los valores necesarios se han obtenido de las hojas de especificaciones de cada uno de los elementos relevantes para el análisis térmico. Los cálculos detallados y los resultados para cada componente se presentan en la Figura 3.8 de este documento.

Thermal Analysis					
COMPONENT	**CONDITIONS**	**POWER(Watts)**	**Max Power(Watts)**	**Theta_ja (°C/W)**	**Temperature (°C above ambient temp)**
LPC2106	(@60MHz)	0.072			
	Idle 7mA	0.0126			
	Power-down 10uA	0.018			
MCP1725-ADJ	3V3-> 1V8	0.1881		163	30.778638
	Quiescent current 220uA	0.000726			
	TOTAL:	0.188826			
TFT 3.98		0.066			
Vibrator DC Motor		0.231			
Auido Amplifier		0.6105		110	67.155
MCP1725-ADJ	3V7-> 3V3	1.0175		163	298.5345
	Quiescent current 220uA	0.814			
	TOTAL:	1.8315			
Red LED		0.0325			
USB-serial		0.075			
MPC738321		0.0075		230	1.725

Figura 3.8. Análisis térmico

3.2. Diseño de la placa de circuito impreso

3.2.1. Importación y creación de huellas

Para la fase de importación de las huellas, se aprovechó en primer lugar la integración nativa de Proteus con los repositorios en línea SnapEDA y SamacSys, así como el servicio gratuito ECAD de Mouser con "Library Loader". Además, en los casos en los que estaba estaba disponible, se añadió automáticamente la geometría 3D de los componentes, de forma que se garantizara la correspondencia exacta con la distribución de pines y los parámetros de soldabilidad recomendados por el proveedor.

Para aquellos elementos que no contaban con huella preexistente en las librerías online, entre los que destacan los botones de las crucetas, el motor vibrador, el conector USB, el diodo de protección, el potenciómetro de volumen y los pines de interconexión de la pantalla TFT, se procedió a modelarlos manualmente en Proteus. Partiendo de las

hojas de especificaciones mecánicas de cada componente, se extrajeron las dimensiones de los pads (longitud, anchura y separación), diámetro de orificios y cotas de los *courtyards*, y se recrearon en el editor de huellas respetando las tolerancias indicadas por el fabricante. Este procedimiento incluyó la definición de zonas de soldadura y máscara de soldadura (*solder mask*) en los casos en los que era necesario.

3.2.2. Posicionado de huellas. Layout PCB

Una vez se dispuso de todas las huellas importadas o creadas manualmente, se adoptó la plantilla mecánica suministrada por el profesor (ver Figura 3.11), que incluye la geometría de la carcasa de la consola, el corte para la pantalla y la división de la PCB en dos segmentos interconectados. A partir de ella, se reprodujo la distribución propuesta en el layout tentativo de los apuntes, con capa superior (Figura 3.9) y capa inferior (Figura 3.10). Se organizó el microcontrolador LPC2106 en el centro de la placa superior para minimizar longitudes de trazado. A su vez, se localizaron los pulsadores de la cruceta y el control de volumen conforme al mecanizado de la carcasa, asegurando siempre un espacio adecuado para el paso de tornillos y conectores. Los componentes que no eran necesarios en esta parte, como por ejemplo la pantalla o la batería, se excluyeron de la PCB.

Durante la colocación, se cuidó que las referencias serigráficas (*silkscreen*) no se superpusieran entre sí ni ocultaran orificios o pads, y se mantuvo un margen mínimo entre el borde de la placa y las huellas críticas. Esta tarea exigió varios reajustes que se hicieron de forma iterativa, ya que al añadir tensiones de prueba y redes de alimentación en la capa interior, se detectaron zonas de congestión de pads que obligaron a reposicionar algunos componentes para respetar las distancias de separación y evitar violaciones de las normas de diseño.

Para el rutado se optó por un trazado manual, ya que el rutado automático que ofrecía Proteus era una peor opción dada la complejidad y densidad de pistas y las conexiones del bus de datos de la pantalla y los buses de control del microcontrolador. Las pistas de tensión se ensancharon deliberadamente para reducir caídas de tensión y mejorar la capacidad de corriente. En torno al LPC2106 fue frecuente recurrir a la capa inferior para aliviar congestionamientos, trazando líneas verticales o *vias jumps* que retornaban a la capa superior cuando el espacio lo permitía.

Una vez completado el enrutamiento de señales, se generaron automáticamente los planos de cobre de masa en ambas capas mediante la herramienta de Proteus, configurando las *clearances* apropiadas para respetar las reglas de separación. A continuación, se ejecutó un *Design Rule Check* (DRC) que confirmó la ausencia de cortocircuitos, pistas demasiado próximas o violaciones de taladro mínimo. Para finalizar, se obtuvieron las capturas de las dos caras de la PCB y la vista 3D del conjunto, que permiten visualizar el encaje mecánico y la adecuación de las alturas de los componentes al interior de la carcasa.

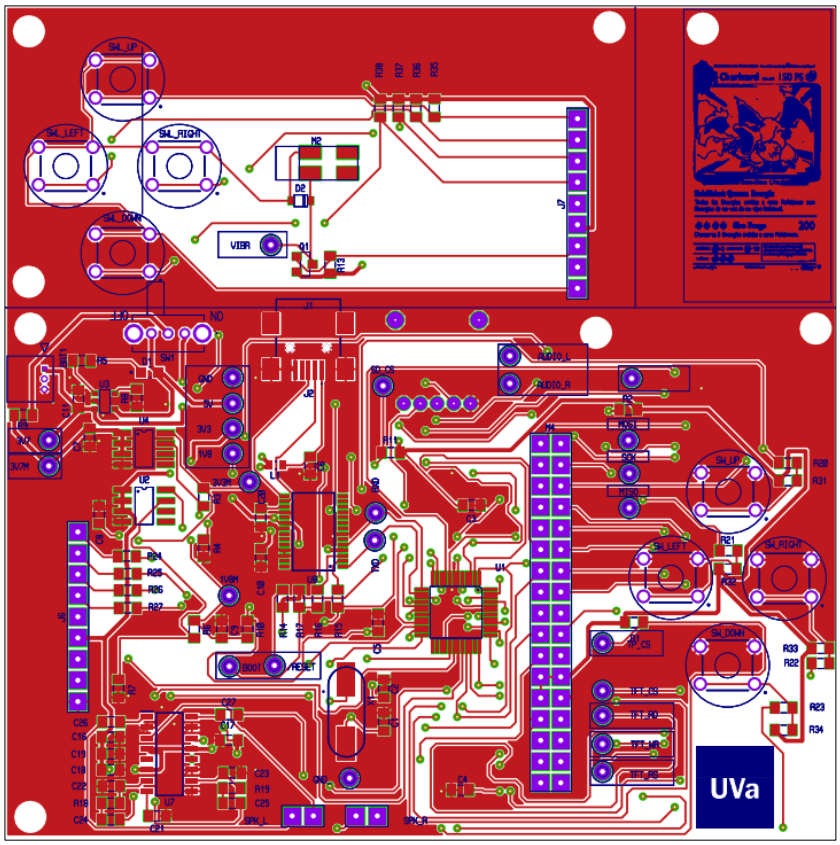


Figura 3.9. Parte superior de la PCB

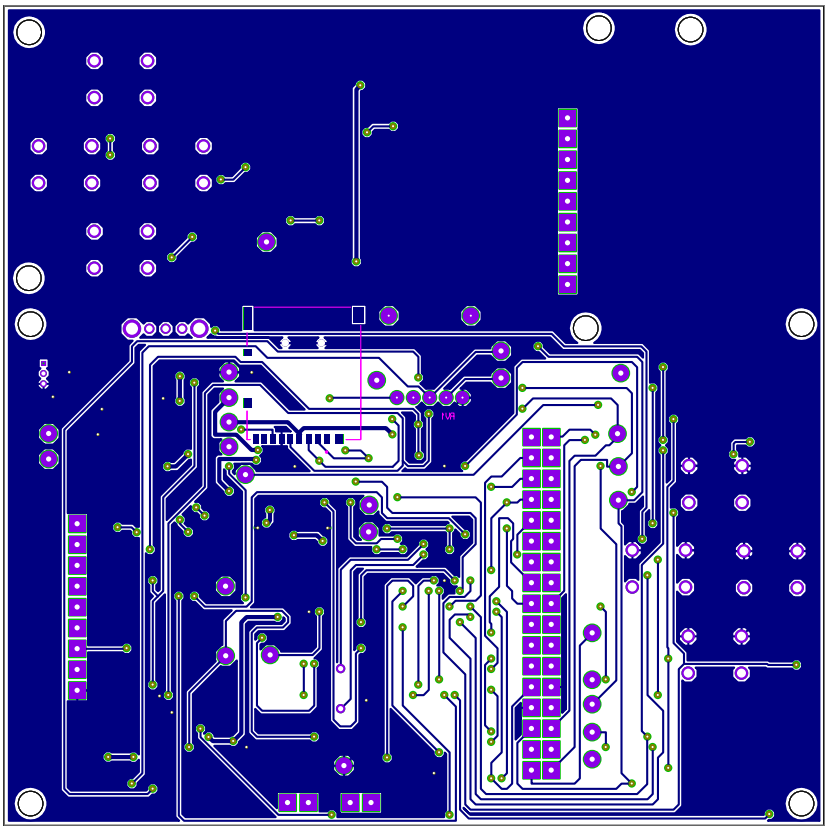


Figura 3.10. Parte inferior de la PCB

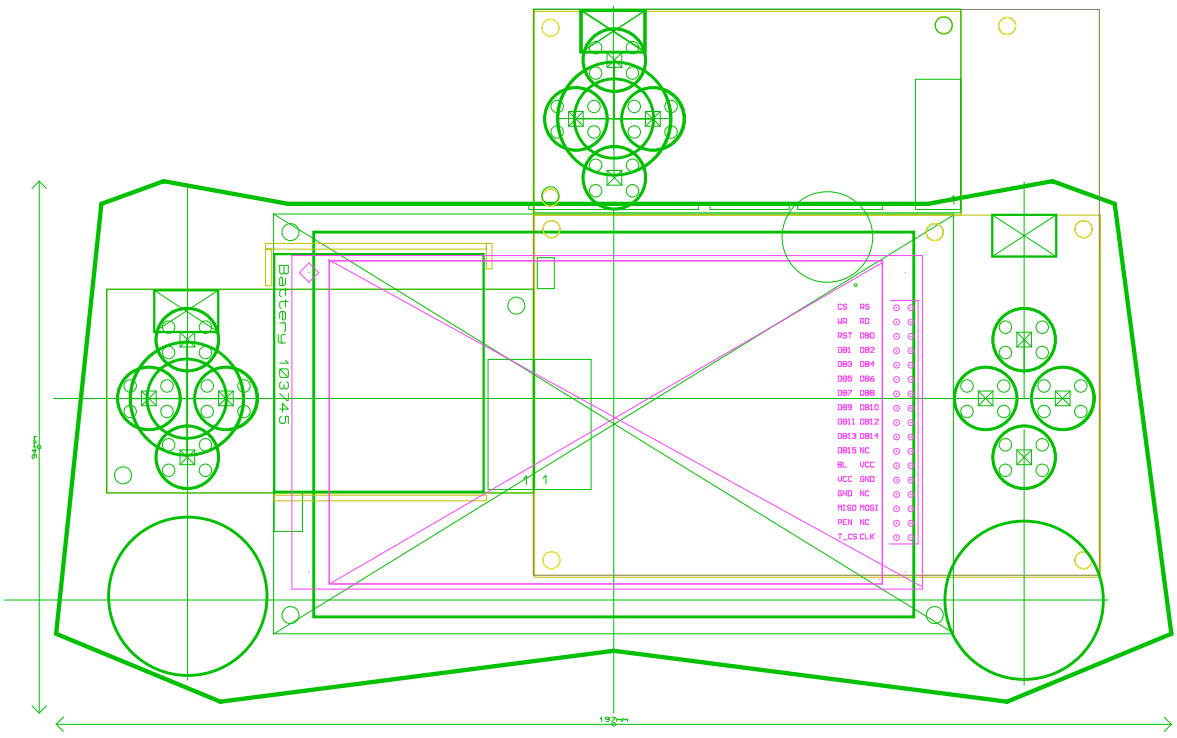


Figura 3.11. Plantilla de mecanizado para la PCB

3.2.3. Generación de ficheros gerber

Los ficheros Gerber constituyen el estándar industrial más utilizado para la fabricación de placas de circuito impreso. Se trata de un conjunto de archivos en formato RS274X cada uno de los cuales describe de forma vectorial una de las capas de la PCB, como por ejemplo cobre superior, cobre inferior, planos de masa, serigrafía, máscara de soldadura, taladros y perforaciones, entre otras. Estos datos incluyen la geometría de pistas, pads, taladros y contornos, así como la información necesaria para el mecanizado del borde de placa. Gracias a su carácter agnóstico respecto al fabricante, los Gerber permiten enviar el diseño a cualquier fábrica de PCBs.

Una vez completado el posicionamiento de huellas, el enrutado de pistas y la verificación mediante DRC, se procedió a realizar un test de pre-producción para comprobar que el diseño sea correcto. Luego, se realizó la exportación de los ficheros Gerber desde Proteus. Mediante el asistente de fabricación se generaron los archivos correspondientes a cada capa, ajustando los parámetros de taladro y definiendo los offsets de capa según las indicaciones del proveedor. Estos ficheros se comprimieron en un único paquete zip, listo para su envío al fabricante, quien se encargará de procesar cada capa y mecanizar la placa de acuerdo con nuestras especificaciones.

3.3. Fabricación y montaje

3.3.1. Recepción y revisión

Las PCBs fueron encargadas a la empresa PCBWay [9] el 6 de marzo de 2025, acompañadas de los ficheros Gerber y las especificaciones mecánicas de la carcasa y la división en dos segmentos de la placa. Tras completar internamente su flujo de producción (pre-ingeniería, corte y laminado, taladrado, metalizado de taladros, grabado, inspección óptica, máscara de soldadura, serigrafía, acabado de superficies, prueba de panel y enrutamiento) la empresa envió las placas hacia el laboratorio el 11 de marzo de 2025.

Al recibir las PCBs, se procedió a una revisión exhaustiva para confirmar la fidelidad del diseño: se comprobó que el contorno y los orificios mecánicos coincidían con la plantilla de la carcasa, que las huellas de conectores y tornillos estaban correctamente alineados y que no existían discontinuidades ni cortocircuitos en las pistas ni restos de máscara en los pads. La serigrafía también fue verificada para asegurarse de que las referencias de componente y las orientaciones permanecían legibles.

3.3.2. Montaje inicial. Soldado de componentes

En primer lugar, la placa rectangular impresa se seccionó cuidadosamente en los dos módulos previstos para la consola mediante una guillotina, siguiendo las líneas de corte marcadas en el diseño mecánico.

A continuación se pasó a realizar las soldaduras, comenzando por los componentes de menor tamaño. Se aplicó primero flux en los pads correspondientes de los condensadores y resistencias SMD, colocándolos con pinzas de punta fina. Tras verificar su posición y continuidad, se incorporaron los circuitos integrados, cuidando particularmente el alineamiento de los pines. Acto seguido, se soldaron los componentes de mayor volumen. (conector USB, potenciómetro de volumen, reguladores de tensión, filas de pines macho y hembra, etc.). Finalmente se fijaron los altavoces y se soldaron sus cables de conexión, se colocó la batería Li-Po de 2000 mAh adquirida en Aliexpress (ver Figura 3.12) en su conector respetando la polaridad y se montó el módulo TFT táctil. El resultado final de esta fase puede observarse en la Figura 3.13.



Figura 3.12. Batería empleada

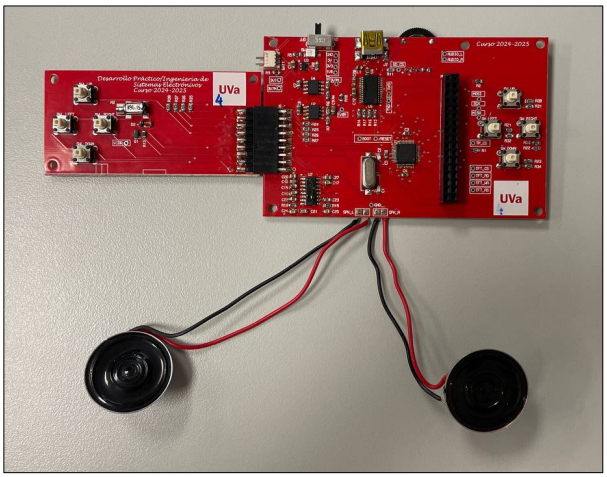


Figura 3.13. PCB finalizada

3.3.3. Verificación y caracterización del hardware

La fase de verificación del hardware comenzó con una comprobación eléctrica sistemática de cada uno de los subsistemas. Se midieron las tensiones en los cuatro rails de alimentación bajo condiciones de reposo y de carga, empleando puntos de test ubicados estratégicamente. A continuación, se verificó la comunicación con el PC mediante el puente entre protocolos USB y UART, comprobando que ordenador reconociese correctamente el microcontrolador en el administrador de dispositivos como un nuevo puerto COM. Seguidamente se comprobó la pantalla TFT y el panel táctil, así como los botones, el vibrador y la amplificación de audio mediante el programa de test ofrecido en los apuntes.

Para caracterizar el comportamiento real del conjunto, se midieron los consumos dinámicos de cada módulo durante escenarios de uso habituales: refresco de pantalla a pleno brillo, lectura continua del táctil, reproducción de audio a volumen máximo y pulsación sostenida de botones con vibración simultánea. Estos ensayos permitieron cuantificar el consumo medio y pico de cada rail. Además, se podría haber registrado la temperatura superficial de los reguladores y del microcontrolador con cámara termográfica, comprobando que las elevaciones se mantenían dentro de los límites previstos en el análisis térmico.

Durante estas pruebas se identificaron y resolvieron varias incidencias menores, la mayoría de ellas relacionadas con contactos mal soldados o componentes defectuosos. Tras corregir estos puntos, se repitieron los ensayos, obteniendo resultados consistentes y acorde a las especificaciones de diseño.

3.3.4. Montaje final

La carcasa exterior de la videoconsola se fabricó mediante impresión 3D de resina, aprovechando la alta resolución de la tecnología SLA para obtener superficies finas y detalles precisos. Tras la impresión, cada pieza pasó por un proceso de lavado en alcohol isopropílico y curado en cabina de luz UV, de modo que se alcanzara la dureza y estabilidad dimensional requeridas. Durante las primeras pruebas se detectó que los botones no encajaban correctamente, por lo que se revisaron las tolerancias en el modelo CAD y se reimprimieron los pulsadores. El resultado puede verse en la Figura 3.14.

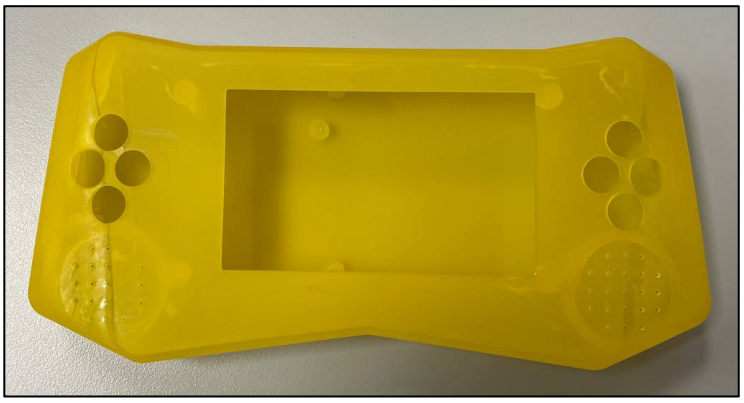


Figura 3.14. Carcasa impresa en 3D

Con la carcasa adecuadamente preparada, se procedió al ensamblaje final. Primero se colocó la PCB principal en su alojamiento, asegurándola mediante un tornillo en el punto diseñado para este fin. A continuación se montaron la pantalla TFT y su panel táctil, encajando el conector hembra correspondiente.

Seguidamente se ubicaron los altavoces en sus soportes laterales, y se instaló la batería Li-Po en su compartimento, anclándola a la carcasa para evitar desplazamientos. Por último, se insertaron los pulsadores impresos en 3D en sus huecos y se acoplaron las dos mitades de la carcasa, verificando que todos los componentes quedaban alojados sin tensiones mecánicas y que la pantalla se veía correctamente. El resultado, presentado en la Figura 3.15, es un ensamblaje compacto y robusto, con alineaciones precisas y un acabado profesional listo para las pruebas funcionales finales.

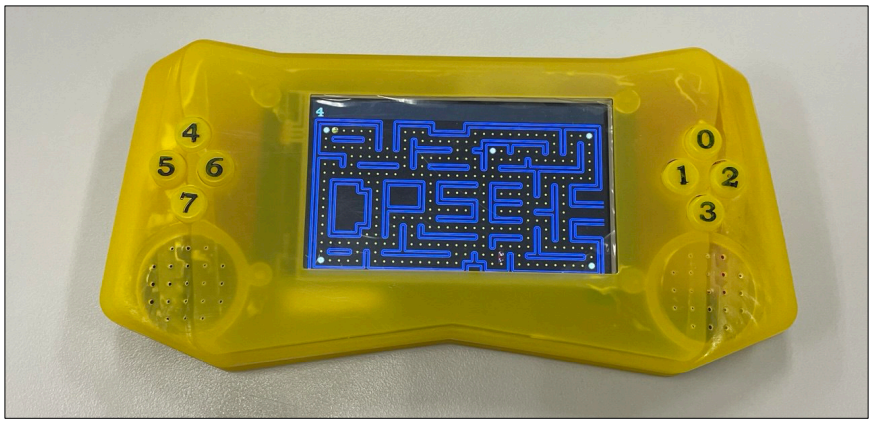


Figura 3.15. Montaje final con Pacman

4. Software

La capa de software de la videoconsola integró el desarrollo de controladores de bajo nivel y la implementación de un videojuego clásico elegido por el grupo. En primer lugar, se crearon drivers en C para gestionar la reproducción de señales de audio PWM a través de los altavoces estéreo y la activación del motor vibrador. A continuación, se adaptó el clásico Pacman a la arquitectura del LPC2106 y al TFT con panel táctil, incorporando algunas variaciones propias.

4.1. Desarrollo del firmware. MCU LPC2106 y simulador

El firmware del microcontrolador ARM7 LPC2106 se programó íntegramente en C, empleando la cadena de herramientas estándar para ARM7. Para validar el comportamiento antes de pasar al hardware real, se adaptó un proyecto alternativo en Proteus con el modelo de simulación del LPC2106 y los componentes virtuales de pantalla, botones, vibrador y amplificador de audio, lo que puede verse en la Figura 4.1. Gracias a esta simulación se pudieron depurar las rutinas existentes y el funcionamiento de los distintos componentes sin necesidad de montaje físico, reduciendo significativamente el ciclo de prueba y error en la fase de integración.

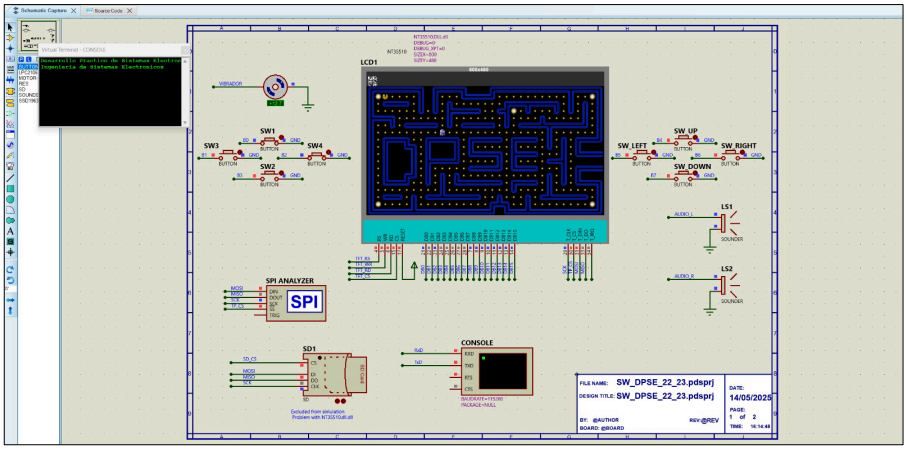


Figura 4.1. Simulador empleando para desarrollar y depurar el firmware

4.2. Controladores

4.2.1. Motor vibrador

El control del motor vibrador se gestiona mediante un driver específico localizado en vibrador.c, que utiliza uno de los temporizadores del microcontrolador LPC2106 para controlar la duración de la vibración.

Función principal

La función principal es VibratorON, que toma un argumento (ms) que especifica la duración en milisegundos durante la cual el motor debe permanecer activo. Los pasos clave que realiza esta función son:

1. Activación del Periférico Timer1. Se asegura de que el periférico Timer1 esté energizado habilitando el bit correspondiente en el registro PCONP (Power Control for Peripherals).
2. Configuración del Timer1. Se resetea el contador del Timer1 (T1TCR = 0x02). El preescalador (T1PR) se establece a 0. El registro T1MR0 se calcula para que el temporizador alcance este valor después de ms milisegundos. Además, el registro T1MCR se configura con el valor 3. Finalmente, se habilita el contador del Timer1 (T1TCR = 1), comenzando la cuenta.
3. Configuración de la Interrupción. Se configura el VIC para que reconozca y gestione la interrupción generada por el Timer1. Se habilita la interrupción del Timer1 (VICIntEnable), se asigna la dirección de la rutina de servicio de interrupción TIMER1_ISR al slot vectorial correspondiente (VICVectAddr2), y se configura y habilita dicho slot (VICVectCntl2).
4. Activación del Vibrador. Se llama a la macro VIBRATOR_ON, responsable de poner a nivel alto el pin GPIO del vibrador.
5. Habilitación de Interrupciones Globales mediante el fragmento de ensamblador.

```
20  void VibratorON(unsigned int ms)
21  {
22   int d;
23   #define PCTIM1 (2)
24   PCONP |= (1<<PCTIM1);      // Power TIMER1 peripheral
25
26    T1TCR =0x02,
27    T1PR = 0;
28    T1MR0 = (60000 * ms);
29    T1MCR = 3;
30    T1TCR = 1;
31
32   #define TIMER1_INT (5)
33   // Interrupt setup
34   VICIntEnable=(1<<TIMER1_INT);                    // TIMER1 interrupt enable
35   VICVectAddr2=(unsigned int)TIMER1_ISR;           // Vector for slot 2
36   VICVectCntl2=(1<<5)|TIMER1_INT;                  // Slot 2 for TIMER1 and enabled
37
38   VIBRATOR_ON;
39
40   asm volatile (
41    " mrs r0,cpsr\n"
42    " bic r0,r0,#0x80\n"
43    " msr cpsr,r0\n"
44   );
45  }
```

Figura 4.2. Función VibratorON

Rutina de servicio de interrupción

Cuando el Timer1 alcanza el valor de T1MR0, se genera una interrupción y se ejecuta la rutina TIMER1_ISR:

1. Limpiar Flag de Interrupción. Se escribe en el registro T1IR para limpiar el flag.
2. Desactivación del Vibrador. Se llama a la macro VIBRATOR_OFF.
3. Detener Timer1. Se escribe 0x02 en T1TCR para resetear y detener el Timer1.
4. Fin de Interrupción (EOI). Se escribe en VICVectAddr para indicar al VIC que la interrupción actual ha sido completamente procesada.

```
11  void TIMER1_ISR(void)
12  {
13   T1IR = 0xFFFF;        // Clear all interrupt notifications
14   VIBRATOR_OFF;
15    T1TCR = 0x02;
16
17   VICVectAddr=0;        // EOI for interrupt controller
18  }
```

Figura 4.3. Función TIMER1_ISR

4.2.2. Audio

La generación de audio se realiza íntegramente por software, utilizando las capacidades de Modulación por Ancho de Pulso (PWM) del microcontrolador LPC2106. El sistema es capaz de reproducir música polifónica de hasta cuatro canales simultáneos y generar efectos de sonido básicos. La base del sistema de audio se encuentra en los ficheros Audio.c y Audio.h, mientras que las partituras se definen en ficheros separados (en nuestro caso pacmania.mod.txt.c).

Reproducción de música (partitura)

La reproducción de una partitura se inicia con la función PARTITURE_On. Esta función recibe un puntero a la partitura (P), el número total de eventos en la partitura (N), y el tempo deseado.

```
327  void PARTITURE_On(unsigned char *P, unsigned int N, unsigned int tempo)
328  {
329  Part=P;
330  Next=N;
331   int ms_negra = 60000/tempo;
332   int ms_semicorchea = ms_negra/4;
333   #define PCTIM0 (1)
334   PCONP |= (1<<PCTIM0);      // Power TIMER0 peripheral
335   T0TCR = 0b00000010;    // Reset
336   T0CTCR= 0b00000000;    // Timer mode
337   T0PR  = 0;         // Prescale 1:1
338   T0PC  = 0;
339   T0MR0 = (PCLK/1000)*ms_semicorchea - 1; // SEMICORCHEA ms  ->aprox. 128 ms
340   T0MCR = 3;         // Interrupt and reset counter
341   T0TCR = 0b00000001;    // Counter enable
342   #define TIMER_INT (4)
343
344   VICIntEnable=(1<<TIMER_INT);                    // TIMER0 interrupt enable
345   VICDefVectAddr=(unsigned int)IRQ_Spurious_Routine; // unvectored IRQs
346   VICVectAddr1=(unsigned int)TIMER0_ISR;             // Vector for slot 1
347   VICVectCntl1=(1<<5)|TIMER_INT;                     // Slot 1 for TIMER0 and enabled
348
349
350  AUDIO_Timer_On();
```

Figura 4.4. Función PARTITURE_On

La partitura es un array bidimensional donde cada fila representa un instante de tiempo (una semicorchea) y cada una de las cuatro columnas contiene el código de la nota a reproducir en uno de los cuatro canales de audio.

```
15  unsigned char PARTITURE[131][4] = {
16
17    C6 ,A5_,A5_,0  ,// |C-601......|A#506......|A#508......|..........
18    0  ,0  ,D5 ,0  ,// |..........|..........|D-508...3FF|..........
19    0  ,C6 ,F5 ,C6 ,// |..........|C-607......|F-508...3..|C-601......
```

Figura 4.5. Fichero con la partitura

Cuando Timer0 interrumpe, se ejecuta TIMER0_ISR. Esta rutina lee las notas correspondientes al instante actual de la partitura para cada uno de los cuatro canales. Para cada nota, calcula un Delta_Fase. Este valor representa cuánto debe avanzar el acumulador de fase de ese canal en cada ciclo de la ISR de PWM para generar la frecuencia correcta. Las frecuencias de las notas están predefinidas en Audio.h y se acceden a través del array NOTE[] en Audio.c.

```
257  void TIMER0_ISR(void)
258  {
259   // Interrupt service routine for changing the partitures' notes
260   // It is called each 128 ms (duration of the sixteenth (semicorchea) note)
261   // It depends on tempo
262   T0IR = 0xFFFF;        // Clear all interrupt notifications
263   // Next note, if any, update delta phases
264    nota1=((Part[cont*4+0]) & 0b00111111);
265    Delta_Fase1= (unsigned int)((256L*NOTE[nota1])/22500); //coger solo los bits del 5:0
266
267
268    nota2=((Part[cont*4+1]) & 0b00111111);
269    Delta_Fase2= (unsigned int)((256L*NOTE[nota2])/22500);
```

Figura 4.6. Función TIMER0_ISR

Síntesis de sonido (PWM)

La generación real de la forma de onda de audio se realiza en PWM0_ISR, que es activada por el temporizador del periférico PWM a una alta frecuencia (57600Hz). La función AUDIO_Timer_On() configura el periférico PWM para este propósito, utilizando dos canales PWM con sus correspondientes pines.

```
287  void AUDIO_Timer_On()
288  {
289   // Interrupt frequency will be 57600 Hz
290   #define PCPWM0 (5)
291   PCONP |= (1<<PCPWM0);      // Power PWM peripheral
292
293   PINSEL0 &=~(0b11<<16);
294   PINSEL0 |= (0b10<<16);     // PWM4/P0.8 as PWM
295
296  // PWM2 configuration
297   PINSEL0 &=~(0b11<<14);
298   PINSEL0 |= (0b10<<14);
299
300   PWMTCR = 0b00000010;       // Reset & PWM mode enabled
301   PWMPR  = 0;                // Prescale = 1/1
302   PWMMCR = 3;                // Interrupt and Reset on match (channel #0)
303   PWMPCR = (1<<12) | (1<<10) ;       // PWM4 single edge  and output enabled
304    //PWM2 single edge and output enabled
305
306   PWMMR0 = 1024-1;     // Interrupt frequency 57600 Hz. Exact 10-bit PWM
307   PWMMR4 = PWMMR0;
308   PWMMR2 = PWMMR0;
309   //PWMLER = (1<< 4) | (1<< 2);      // Copy to duty reg from shadow PWM4
```

Figura 4.7. Función AUDIO_Timer_On

Dentro de PWM0_ISR, se utiliza la técnica de síntesis por tabla de ondas. Una tabla (TABLA_SINUS_WAVE en Audio.c) almacena 256 muestras de un ciclo de una onda senoidal.

```
26    const short TABLA_SINUS_WAVE[]={
27        0,   12,   25,   37,   50,   62,   75,   87,
28       99,  112,  124,  136,  148,  160,  172,  184,
29      195,  207,  218,  230,  241,  252,  263,  273,
```

Figura 4.8. Tabla de senos

Para cada uno de los cuatro canales de música, la ISR:

1. Obtiene el valor actual de la muestra de la tabla utilizando un acumulador de fase (Fase1, Fase2, etc.). El índice en la tabla se obtiene desplazándolo (Fase1 >> NT1, donde NT1 es 8, ya que 2^8 = 256).
2. Incrementa el acumulador de fase en la cantidad Delta_Fase correspondiente (calculada en TIMER0_ISR).
3. Atenúa la muestra obtenida. En nuestro caso d1 = (d1 >> 2).
4. Mezcla las muestras de los canales: d1 y d2 se suman para el canal izquierdo y d3 y d4 para el canal derecho.
5. Si hay un efecto de sonido activo, su muestra se suma a ambos canales.
6. Los valores finales se saturan para no exceder el rango del PWM y se escriben en los registros de comparación PWMMR4 (izquierdo) y PWMMR2 (derecho), ajustados con un offset para centrar la onda en el rango del PWM (0-1023).
7. Se activa el latch (PWMLER) para que los nuevos valores de ciclo de trabajo tomen efecto.

```
155   void PWM0_ISR(void)
156   {
157     static int d1,d2,d3,d4, dtotalIzq, dtotalDch;
158       PWMIR = 0xFFFF;    // Clear all interrupt notifications
159     static  int NT1=8; //256 elementos de la tabla
160
161   // Get data
162     d1 = TABLA1[Fase1>>NT1];
163     d2 = TABLA1[Fase2>>NT1];
164     d3 = TABLA1[Fase3>>NT1];
165     d4 = TABLA1[Fase4>>NT1];
166
167   // Attenuate
168     d1=(d1>>2);
169     d2=(d2>>2);
170     d3=(d3>>2);
171     d4=(d4>>2);
172
173   // Increment phases
174       Fase1+=Delta_Fase1;
175       Fase2+=Delta_Fase2;
176       Fase3+=Delta_Fase3;
177       Fase4+=Delta_Fase4;
```

Figura 4.9. Función PWM0_ISR

4.3. Videojuego: Pacman clásico

El código fuente principal del videojuego se localiza en el fichero main.c. Este fichero contiene la lógica completa del juego, desde la inicialización del mapa y los personajes, hasta la gestión de las entradas del usuario, el movimiento de Pac-Man y los fantasmas, la detección de colisiones, y la actualización de la pantalla y el HUD con la puntuación y las vidas restantes. El diseño del mapa personalizado ha sido modificado para incluir las siglas de la asignatura DPSE como parte de los muros del laberinto. Los elementos gráficos, como los sprites de Pac-Man, los fantasmas en sus diferentes estados y colores, las monedas, y los muros del laberinto, fueron exportados al formato XPM (X PixMap). Cada sprite se trabajó con un tamaño de 24x24 píxeles.

La estructura general del código se basa en un bucle principal que se ejecuta continuamente. Dentro de este bucle, se procesan las entradas de los botones, se actualiza la lógica de movimiento de los personajes, se comprueban las colisiones y se redibuja la pantalla según sea necesario. Se ha puesto especial énfasis en lograr un movimiento fluido y sin parpadeos para los personajes.

Inicialización del juego

Al comenzar el juego, la función initGame se encarga de establecer el estado inicial. Esto incluye copiar el diseño del mapa base, originalMap, a una matriz de trabajo map. originalMap es una constante que define la estructura del laberinto, incluyendo la posición inicial de Pac-Man, los fantasmas, las monedas y los power-ups. Durante la inicialización, se identifican las posiciones de Pac-Man y los fantasmas, se guardan las posiciones originales de los fantasmas para su reaparición (origGhostX, origGhostY), se cuentan las monedas totales (totalCoins) y se configuran las ubicaciones de teletransporte (teleR, teleC).

```
284  static void initGame(void)
285  {
286      int r, c, g;
287
288      for (r = 0; r < MAP_ROWS; r++) {
289          for (c = 0; c < MAP_COLS; c++) map[r][c] = originalMap[r][c];
290          map[r][MAP_COLS] = '\0';
291      }
292      teleCount = 0;
293      for (r = 0; r < MAP_ROWS; r++) {
294          for (c = 0; c < MAP_COLS; c++) {
295              if (map[r][c] == FDOT && teleCount < 2) {
296                  teleR[teleCount] = r;
297                  teleC[teleCount] = c;
298                  teleCount++;
299              }
300          }
301      }
```

Figura 4.10. Función initGame

Control y movimiento de Pacman

El movimiento de Pac-Man es gestionado por la función movePacman. Esta función primero lee la entrada del usuario mediante GetAction(), que devuelve un valor

indicando qué botones direccionales están presionados. Esta entrada se traduce a una desiredDir (dirección deseada). La función canMoveDir verifica si la casilla en la dirección especificada es un muro (wallAt).

```
480  static void movePacman(void)
481  {
482      int keys = GetAction();
483      if      (keys & UP)    desiredDir = DIR_UP;
484      else if (keys & DOWN)  desiredDir = DIR_DOWN;
485      else if (keys & LEFT)  desiredDir = DIR_LEFT;
486      else if (keys & RIGHT) desiredDir = DIR_RIGHT;
487
488      /* Sólo en cruces chequea monedas y posible cambio de dirección */
489      bool onTileX = ((pacPixX - X_OFFSET) % TW) == 0;
490      bool onTileY = ((pacPixY - Y_OFFSET) % TW) == 0;
491
492      if (onTileX && onTileY) {
493          int cx = (pacPixX - X_OFFSET) / TW;
494          int cy = (pacPixY - Y_OFFSET) / TW;
495
```

Figura 4.11. Función movePacman

La animación suave del movimiento se logra en animatePacmanMove. En lugar de saltar de una casilla a otra, Pac-Man se mueve STEP_PX píxeles en cada iteración de la animación hasta alcanzar la casilla destino. En cada paso de esta animación, se restaura el fondo que Pac-Man ocupaba (restoreBackground) y se dibuja el sprite de Pac-Man en la nueva posición. La boca de Pac-Man alterna entre abierta y cerrada (pacMouthOpen, pacMouthCounter) para simular el acto de comer.

```
457  static void animatePacmanMove(int nx, int ny)
458  {
459      int dx    = (nx > pacPixX) ? STEP_PX : (nx < pacPixX ? -STEP_PX : 0);
460      int dy    = (ny > pacPixY) ? STEP_PX : (ny < pacPixY ? -STEP_PX : 0);
461      int steps = (abs(nx - pacPixX) + abs(ny - pacPixY)) / STEP_PX;
462      int i;
463      for (i = 0; i < steps; i++) {
464          restoreBackground(pacPixX, pacPixY);
465          pacPixX += dx;
466          pacPixY += dy;
467          /* alterna boca */
468          if (++pacMouthCounter >= PAC_MOUTH_TOGGLE_STEPS) {
469              pacMouthOpen    = !pacMouthOpen;
470              pacMouthCounter = 0;
471          }
472          /* dibuja según dirección actual */
473          int frame = getPacmanFrame(currentDir);
474          XPM_PintaAtxNyN(pacPixX, pacPixY, frame, pacman);
475          if (PAC_DELAY_MS) _delay_ms(PAC_DELAY_MS);
476      }
477  }
```

Figura 4.12. Función animatePacmanMove

Control y movimiento de los fantasmas

Los fantasmas son controlados por la función moveGhosts. Cada fantasma se representa mediante una estructura Ghost que almacena su posición, dirección actual, el tipo de casilla que ocupa en el mapa (under) y su tipo (color). En la implementación actual, el movimiento de los fantasmas es predominantemente aleatorio. Si un fantasma no tiene una dirección asignada (DIR_NONE) o si se cumple una condición aleatoria

((rand() & 7) == 0), se intenta seleccionar una nueva dirección válida al azar. La liberación de los fantasmas desde su "casa" es escalonada, controlada por ghostReleased y GHOST_RELEASE_INTERVAL_STEPS.

```
652  static void moveGhosts(void)
653  {
654      static const Direction dirs[4] = { DIR_UP, DIR_DOWN, DIR_LEFT, DIR_RIGHT };
655      int i, t;
656      for (i = 0; i < GHOST_COUNT; i++) {
657          if (!ghostReleased[i]) continue;
658          Ghost *g = &ghosts[i];
659
660          /* A veces elige nueva dirección al azar */
661          if (g->dir == DIR_NONE || (rand() & 7) == 0) {
662              for (t = 0; t < 8; t++) {
663                  Direction d = dirs[rand() % 4];
664                  int nx = g->x + (d==DIR_RIGHT) - (d==DIR_LEFT);
665                  int ny = g->y + (d==DIR_DOWN)  - (d==DIR_UP);
666                  if (!wallAt(ny, nx)) {
667                      g->dir = d;
668                      break;
669                  }
670              }
671          }
```

Figura 4.13. Función moveGhosts

Al igual que Pac-Man, el movimiento de los fantasmas es animado píxel a píxel por animateGhostMove. Esta función también se encarga de restaurar la casilla del mapa que el fantasma abandona y actualizar la matriz map para reflejar la nueva posición del fantasma. La función getGhostFrame selecciona el sprite del fantasma, teniendo en cuenta su orientación (hacia dónde mira Pac-Man) y el estado de sus ojos (parpadeo gestionado por ghostsEyeOpen). Si Pac-Man está en modo powerMode (tras comer un power-up), los fantasmas se dibujan con un sprite especial de vulnerabilidad.

Renderizado del mapa y HUD

El dibujo inicial del mapa se realiza con dibuja_mapa, que itera sobre la matriz map y llama a drawTile para cada celda. drawTile selecciona el sprite XPM correspondiente al carácter de la celda (muro, moneda, vacío, etc.) y lo renderiza en la pantalla usando XPM_PintaAtxNyN. El HUD, que muestra las vidas y la puntuación, se actualiza mediante dibuja_hud solo cuando es necesario (indicado por la bandera hudDirty = true). Esta función utiliza TFT_print_xNyN para mostrar el texto. La conversión de los valores numéricos de vidas y puntuación a cadenas de texto se realiza con una función auxiliar ft_itoa que hemos programado específicamente para el HUD.

```
357  static void drawTile(int r, int c)
358  {
359      int px = X_OFFSET + c * TW;
360      int py = Y_OFFSET + r * TW;
361      switch (map[r][c]) {
362          case WALL_UP:       XPM_PintaAtxNyN(px, py, 49, pacman);  break;
363          case WALL_RIGHT:    XPM_PintaAtxNyN(px, py, 62, pacman);  break;
364          case WALL_LEFT:     XPM_PintaAtxNyN(px, py, 60, pacman);  break;
365          case WALL_DOWN:     XPM_PintaAtxNyN(px, py, 73, pacman);  break;
366          case CORNER_UPR:    XPM_PintaAtxNyN(px, py, 50, pacman);  break;
367          case CORNER_UPL:    XPM_PintaAtxNyN(px, py, 48, pacman);  break;
368          case CORNER_DOWNR:      XPM_PintaAtxNyN(px, py, 74, pacman);  break;
369          case CORNER_DOWNL:      XPM_PintaAtxNyN(px, py, 72, pacman);  break;
```

Figura 4.14. Función drawTile

Detección de colisiones y lógica del juego

La función checkCollisions se encarga de verificar si Pac-Man ha colisionado con un fantasma. Compara la posición de Pac-Man (centro de su sprite) con la posición de cada fantasma activo. Si Pac-Man está en modo powerMode, el fantasma colisionado es "comido" y enviado de vuelta a su punto de origen. Si no, Pac-Man pierde una vida. Si las vidas llegan a cero, el juego termina. El juego también termina victoriosamente si totalCoins llega a cero.

```
687  static void checkCollisions(void)
688  {
689      int tx = (pacPixX - X_OFFSET + TW / 2) / TW;
690      int ty = (pacPixY - Y_OFFSET + TW / 2) / TW;
691      int g;
692      for (g = 0; g < GHOST_COUNT; g++) {
693          if (!ghostReleased[g]) continue;
694          if (tx == ghosts[g].x && ty == ghosts[g].y) {
695              lives--; hudDirty = true;
696              if (lives > 0) {
697                  restoreBackground(pacPixX, pacPixY);
698                  initGame(); dibuja_mapa(map); dibuja_hud();
699              }
700              return;
701          }
702      }
703  }
```

Figura 4.15. Función checkCollisions

Bucle principal

El main contiene el bucle principal del juego. Tras inicializar el sistema, la pantalla TFT, el panel táctil y el audio (reproduciendo música de fondo), entra en un ciclo while(1) que constituye el núcleo del juego. Este bucle gestiona el flujo del juego: actualiza el estado de Pac-Man y los fantasmas, maneja el modo power-up, libera fantasmas progresivamente, comprueba colisiones y actualiza la interfaz de usuario. La inclusión de VibratorON(200) al liberar un fantasma proporciona una realimentación háptica al jugador.

```
719  int main(void)
720  {
721      SCS = 1;
722      FIODIR  = 0b11111111111111111111111111011101;
723      FIOSET  = 0xFFFFFFFF;
724      PINSEL0 = 0x00029505;
725      PINSEL1 = 0x00000000;
726      _delay_ms(100);
727
728      _printf("Desarrollo Practico de Sistemas Electronicos\r\n"
729              "Ingenieria de Sistemas Electronicos\r\n");
730
731      AMPLIF_ON;
732      start_pacmania_music();
733
734      TFT_Init();
735      TP_Init();
736
737      TFT_DrawFillSquareS(0, 0, LENX, LENY, TFT_Color(0, 0, 0));
738
739      initGame();
740      dibuja_mapa(map);
741      dibuja_hud();
742      start_pacmania_music();
743
```

Figura 4.16. Bucle principal

5. Conclusiones y líneas futuras

En este proyecto se ha demostrado la viabilidad de materializar una videoconsola portátil completa partiendo de requisitos funcionales básicos hasta la puesta en marcha de un prototipo plenamente operativo. A nivel de hardware, el LPC2106 ha permitido gestionar simultáneamente el bus de datos de la pantalla TFT, el panel táctil, la activación del motor vibrador y la reproducción de audio estéreo, todo ello con un consumo y disipación térmica controlados gracias a los análisis realizados. El proceso de diseño en Proteus, la generación de ficheros Gerber, la fabricación en PCBWay y el montaje manual han validado las metodologías y herramientas empleadas. Por otro lado, el firmware en C incorpora drivers para cada periférico y una versión adaptada de Pacman que aprovecha la resolución de la pantalla y la interfaz de botones, cumpliendo los objetivos de funcionalidad y robustez planteados al inicio de este trabajo.

Como líneas de mejora, conviene explorar la optimización del consumo en modos de reposo, implementando rutinas de bajo consumo o suspensión de periféricos para prolongar la autonomía. En el plano ergonómico y funcional, resultaría interesante dotar a la consola de conectividad inalámbrica (Bluetooth o Wi-Fi) para conexiones con el PC e incluso partidas multijugador, así como ampliar la biblioteca de videojuegos e incluir un sistema de menús y actualizaciones OTA. Finalmente, la adaptación del diseño a una placa única (sin dividir la PCB), la posibilidad de cargar juegos desde la tarjeta microSD o la integración de un joystick analógico permitirían elevar la experiencia de usuario y abrirían la puerta a proyectos posteriores de carácter educativo.

Referencias

[1] «Curso: DESARROLLO PRACTICO DE SISTEMAS ELECTRONICOS (1-211-460-45041-1-2024) | CampusVirtualUVa». Accedido: 9 de mayo de 2025. [En línea]. Disponible en: https://campusvirtual.uva.es/course/view.php?id=61484

[2] «DPSE_2024_2025». Accedido: 9 de mayo de 2025. [En línea]. Disponible en: https://uvaes-my.sharepoint.com/personal/jesus_hernandez_mangas_uva_es/_layouts/15/Doc.aspx?sourcedoc={a5b3b519-e5ab-4e32-8639-d6d952ecf674}&action=view&wd=target%28HARDWARE.one%7C62fd1678-cbf0-4822-81cf-f5183a93a2f8%2F5.1%20Fabricaci%C3%B3n%20PCB%20Reglas%20de%20dise%C3%B1o%7C1a1457ca-5ee3-4924-bec3-312ee26b9471%2F%29&wdorigin=NavigationUrl

[3] «UM10275 LPC2104/2105/2106 User manual», vol. 2009, 2009.

[4] «PCB Design and Circuit Simulator Software - Proteus». Accedido: 9 de mayo de 2025. [En línea]. Disponible en: https://www.labcenter.com/

[5] B. S. s.r.o, «GanttProject: free project management tool for Windows, macOS and Linux», GanttProject. Accedido: 6 de mayo de 2025. [En línea]. Disponible en: https://www.ganttproject.biz

[6] «Let’s make your design a snap.» Accedido: 9 de mayo de 2025. [En línea]. Disponible en: https://www.snapeda.com

[7] «Home», SamacSys. Accedido: 9 de mayo de 2025. [En línea]. Disponible en: https://www.samacsys.com/

[8] «Farnell España - Distribuidor de componentes electrónicos». Accedido: 7 de mayo de 2025. [En línea]. Disponible en: https://es.farnell.com/

[9] «China PCB Prototype & Fabrication Manufacturer - PCB Prototype the Easy Way». Accedido: 8 de mayo de 2025. [En línea]. Disponible en: https://www.pcbway.com/